apiSchema() Reference Manual

Complete guide to defining `apiSchema()` on your Eloquent models for use with the UiApi package. This document covers every key, its purpose, accepted values, defaults, and what happens when it is missing.

Table of Contents

- Overview
- Minimal Example
- Column Definition Keys — Full Reference
  - hidden
  - key
  - label
  - type
  - lang
  - sortable
  - displayType
  - inputType
  - formField
  - inlineEditable
  - fieldComponent
  - validationRule
- Display Configuration
  - displayType Values
  - chip — Chip Display
  - displayProps — Generic Display Properties
  - How displayType Resolution Works
- Select & Dropdown Configuration
  - The select Key
  - Mode: self
  - Mode: relation
  - Mode: url
  - Common Select Sub-Keys
- Language Support

- How lang Filtering Works
- Columns Without lang
- Language-Specific Columns
- Relation Columns
  - relationLabel
- Computed Attributes
- Complete Annotated Example
- Quick Reference Table

Overview

Every model that participates in the UiApi system must expose an `apiSchema()` method. This method returns an array with a `columns` key that maps each column (or computed attribute) to a **column definition** — a set of keys describing how the column should behave across the entire UI stack: tables, forms, filters, data links, and headers.

```
Model::apiSchema()
  └── 'columns'
        ├── 'id'          => [ ...column definition... ]
        ├── 'name'        => [ ...column definition... ]
        ├── 'status'      => [ ...column definition... ]
        └── 'country_id'  => [ ...column definition... ]
```

The `ComponentConfigService` (CCS) reads these definitions to:

1. **Build table headers** — title, sortability, display rendering
2. **Build form fields** — input types, select dropdowns, labels
3. **Build filter panels** — filter type, select options, search fields
4. **Build data links** — auto-generated API URLs with correct columns and relations
5. **Enforce language support** — show/hide columns based on requested language

Your model **must** extend `Ogp\UiApi\Models\BaseModel` (or have the equivalent scopes). The package resolves models by checking `Ogp\UiApi\Models\{Model}` first, then `App\Models\{Model}`.

Minimal Example

The smallest working model:

```php
<?php

namespace App\Models;

use Ogp\UiApi\Models\BaseModel;

class Book extends BaseModel
{
    public function apiSchema(): array
    {
        return [
            'columns' => [
                'id' => [
                    'hidden' => true,
                    'key' => 'id',
                    'label' => ['en' => 'ID'],
                    'type' => 'number',
                    'displayType' => 'text',
                    'lang' => ['en'],
                    'sortable' => true,
                ],
                'title' => [
                    'hidden' => false,
                    'key' => 'title',
                    'label' => ['en' => 'Title'],
                    'type' => 'string',
                    'displayType' => 'text',
                    'lang' => ['en'],
                    'sortable' => true,
                    'inputType' => 'textField',
                    'formField' => true,
                ],
            ],
        ];
    }
}
```

> **Tip:** Even though some keys have defaults, being explicit makes the schema self-documenting and avoids surprises when the CCS processes it.

Column Definition Keys — Full Reference

Each column in the `columns` map is a key-value pair where the **key** is the column name (matching the database column or a computed attribute) and the **value** is an associative array of the following keys.

---

hidden

Property	Value
Type	`bool`
Required	No
Default	`false`

Controls whether the column is visible in table headers. When `true`, the column is excluded from the generated headers array (unless `includeHiddenColumnsInHeaders` is enabled on the CCS). The column can still be requested via `columns` query parameter and will appear in API data, but the UI table will not show a header for it.

Use cases:

- Primary keys (`id`) — needed for data operations but not shown in tables
- Foreign keys (`country_id`) — the related display column is shown instead
- Internal timestamps (`updated_at`)

```
'id' => [
    'hidden' => true,  // No table header generated
    // ...
],
```

If missing: Defaults to `false` — the column is visible.

---

key

Property	Value
Type	`string`
Required	No (but recommended)
Default	The column map key (e.g., `'id'`, `'status'`)

The identifier used in the output payload. When CCS builds headers, form fields, or filters, it reads this value to set the `value` (in headers) or `key` (in fields/filters) property.

```
'ref_num' => [
    'key' => 'ref_num',  // Appears as header.value = 'ref_num'
    // ...
],
```

If missing: Falls back to the array key itself. In most cases, you should set `key` to match the array key for clarity.

---

label

Property	Value
Type	`string` or `array` (localized map)
Required	No
Default	Auto-generated from the column name (e.g., `ref_num` → `"Ref Num"`)

The human-readable name shown in table headers, form labels, and filter labels.

String form (single language):

```
'label' => 'Reference Number',
```

Localized map (recommended for multi-language apps):

```
'label' => ['en' => 'Reference Number', 'dv' => 'ﷺ ﷺ'],
```

Resolution order when the CCS picks the label for a given request language (e.g., `lang=en`):

1. `label[$lang]` — the exact requested language
2. `label['en']` — English fallback
3. `label['dv']` — Dhivehi fallback
4. First non-empty value in the array
5. Auto-generated from column name: `Str::title(str_replace('_', ' ', $field))`

If missing: The column name is title-cased automatically (`first_name_eng` → `"First Name Eng"`).

---

type

Property	Value
Type	`string`
Required	No
Default	`'string'`

The data type of the column. Affects:

- **Default inputType inference** — if `inputType` is not set, `type` determines the default form input
- **Default filter type** — date types get a `Date` filter, others get `Text`
- Passed through to headers as `header.type` for frontend rendering logic

Accepted values:

Value	Default inputType	Default filter type	Notes
`'string'`	`textField`	`Text`	Most common

Value	Default inputType	Default filter type	Notes
`'number'`	`numberField`	`Text`	Numeric fields
`'integer'`	`numberField`	`Text`	Alias for number
`'boolean'`	`checkbox`	`Checkbox`	True/false fields
`'date'`	`datepicker`	`Date`	Date only
`'datetime'`	`datepicker`	`Date`	Date + time
`'json'`	`textField`	`Text`	JSON data

```
'total_pages' => [
    'type' => 'number',
    // inputType will default to 'numberField' if omitted
],
```

If missing: Treated as `'string'`. Form fields default to `textField`, filters default to `Text`.

---

lang

Property	Value
Type	`array` of language codes
Required	No (but strongly recommended)
Default	Column is included in all languages

Restricts which UI languages this column appears in. When the CCS processes a request with `?lang=en`, only columns whose `lang` array includes `'en'` are included in headers, data links, and filters.

```
// Available in both English and Dhivehi
'ref_num' => [
    'lang' => ['en', 'dv'],
],

// Only shown in English views
'first_name_eng' => [
    'lang' => ['en'],
],
```

```
// Only shown in Dhivehi views
'first_name_div' => [
    'lang' => ['dv'],
],
```

If missing: The column is treated as available in **all** languages — it will appear regardless of the `lang` request parameter.

See the Language Support section for detailed behavior.

---

sortable

Property	Value
Type	`bool`
Required	No
Default	`false`

Whether the column can be sorted via the `?sort=` query parameter. When `true`, the generated header includes `sortable: true`, allowing the frontend to enable sort controls.

```
'created_at' => [
    'sortable' => true,
],
```

If missing: Defaults to `false` — the column is not sortable.

---

displayType

Property	Value
Type	`string`

Property	Value
Required	No
Default	None (omitted from header)

Tells the frontend **how to render** the column value in tables. The CCS passes this value through to the header payload.

Accepted values:

Value	Description	Requires additional config?
`'text'`	Plain text display	No
`'englishText'`	Text rendered in LTR/English style	No
`'chip'`	Colored chip/badge	Yes — needs `chip` or `displayProps` key
`'date'`	Formatted date display	No
`'checkbox'`	Checkbox/boolean display	No (optionally `displayProps`)
`'select'`	Select-style display	Yes — needs `select` or `displayProps` key

Important: When `displayType` is `'chip'` or `'select'`, the CCS looks for a matching sub-key (e.g., a `chip` key) or a `displayProps` key. If neither exists, the ViewConfigValidator will emit a warning and display may fall back to plain text.

See Display Configuration for detailed examples.

If missing: The `displayType` key is omitted from the header output. The frontend should handle this gracefully (typically rendering as plain text).

---

inputType

Property	Value
Type	`string`
Required	No

Property	Value
Default	Inferred from `type`

Determines the form input component used when this column appears in a form. Also influences filter rendering — the CCS uses `inputType` to decide the filter type token.

Accepted values:

Value	Renders as	Typical `type`
`'textField'`	Text input	`string`
`'numberField'`	Number input	`number`, `integer`
`'datepicker'`	Date picker	`date`, `datetime`
`'dateField'`	Date field (alias)	`date`
`'select'`	Dropdown select	Any — requires `select` or `filterable` config
`'checkbox'`	Checkbox toggle	`boolean`

When `inputType` is `'select'`: The CCS looks for a `select` key (falling back to `filterable`) to build the dropdown options. If neither exists, the ViewConfigValidator raises an error.

Default inference (when `inputType` is omitted):

`type` value	Inferred `inputType`
`'string'`	`'textField'`
`'number'`	`'numberField'`
`'boolean'`	`'checkbox'`
`'date'`	`'datepicker'`
anything else	`''` (empty)

```
    'summary' => [
        'type' => 'string',
        // inputType defaults to 'textField'
    ],
    'status' => [
        'type' => 'string',
        'inputType' => 'select',  // Override: use dropdown instead of
text input
```

```
        'select' => [ /* ... */ ],
    ],
```

If missing: Inferred from `type`. If `type` is also missing, defaults to empty string.

---

formField

Property	Value
Type	`bool`
Required	No
Default	`false`

Controls whether this column is included in auto-generated form fields. When the CCS builds form payloads (via `buildFormFieldsFromSchema`), only columns with `formField: true` are included.

```
    'created_at' => [
        'formField' => false,  // Not editable via forms
    ],
    'summary' => [
        'formField' => true,   // Appears in create/edit forms
    ],
```

If missing: Defaults to `false` — the column is **not** included in form field output. This means if you forget to set `formField: true`, the column will not appear in auto-generated forms.

---

inlineEditable

Property	Value
Type	`bool`
Required	No

Property	Value
Default	`false`

Indicates whether the column can be edited directly within a table row (inline editing). The CCS passes this through as `header.inlineEditable` for the frontend.

```
    'status' => [
        'inlineEditable' => true,  // Can be edited in-place in the
table
    ],
```

If missing: Defaults to `false` — inline editing is disabled for this column.

> **Note:** `editable` is treated as an alias for `inlineEditable` in columnCustomizations. In the apiSchema, use `inlineEditable`.

---

fieldComponent

Property	Value
Type	`string`
Required	No
Default	None

Pass-through metadata specifying which frontend component should render this field in forms. This value is not consumed by the CCS directly — it is included in the schema for the frontend to read.

```
    'id' => [
        'fieldComponent' => 'textInput',
    ],
```

Accepted values: Depends on your frontend component library. Common values seen in this codebase: `'textInput'`.

If missing: Omitted from the schema. The frontend should fall back to a default component based on `inputType`.

---

validationRule

Property	Value
Type	`string`
Required	No
Default	None

Pass-through metadata containing Laravel-style validation rules as a pipe-delimited string. This value is not consumed by the CCS — it is included in the schema for the frontend to read (e.g., for client-side validation hints).

```
'first_name_eng' => [
    'validationRule' => 'required|string|max:255',
],
```

> **Important:** This is separate from the model's `baseRules()` / `rulesForCreate()` / `rulesForUpdate()` methods, which handle actual server-side validation. The `validationRule` key here is informational metadata for the frontend only.

If missing: Omitted from the schema. No effect on server-side validation.

Display Configuration

displayType Values

The `displayType` key tells the frontend how to render a column value. Here is how each value behaves and what additional configuration it may need:

`'text'`

Plain text rendering. No additional config needed.

```
'summary' => [
    'displayType' => 'text',
],
```

`'englishText'`

Text rendered with left-to-right (English/Latin) styling. Useful in RTL applications where some columns contain English content.

```
'first_name_eng' => [
    'displayType' => 'englishText',
],
```

`'date'`

Formatted date/datetime rendering.

```
'submitted_at' => [
    'type' => 'datetime',
    'displayType' => 'date',
],
```

`'checkbox'`

Boolean display as a checkbox. Optionally styled with `displayProps`.

```
'is_in_custody' => [
    'type' => 'boolean',
    'displayType' => 'checkbox',
    'displayProps' => [
        'label' => ' ',
        'color' => 'primary',
    ],
],
```

`'chip'`

Colored badge/chip rendering — requires a `chip` key or `displayProps`. See below.

`'select'`

Select-style display — requires a matching `select` key or `displayProps`.

---

chip — Chip Display

When `displayType` is `'chip'`, you must provide a `chip` key (or `displayProps`) that maps each possible **column value** to a display configuration object.

Structure:

```
    'chip' => [
        '<value1>' => [
            'label' => ['en' => 'English Label', 'dv' => 'Dhivehi
Label'],
            'color' => '<color>',
            'prependIcon' => '<icon-name>',  // optional
        ],
        '<value2>' => [
            'label' => ['en' => '...', 'dv' => '...'],
            'color' => '<color>',
        ],
    ],
```

Chip entry keys:

Key	Type	Required	Description
`label`	`string` or `array`	Yes	Display text. Localized map recommended.
`color`	`string`	Yes	Chip color (e.g., `'primary'`, `'success'`, `'error'`, `'warning'`, `'secondary'`).
`prependIcon`	`string`	No	Icon name displayed before the label.

The chip map key must match the actual data value stored in the database. For string columns, use the string value; for numeric columns, use the integer value.

Full example — string values:

```
    'status' => [
        'hidden' => false,
        'key' => 'status',
        'label' => ['en' => 'Status', 'dv' => 'ڪُڊُهُ'],
        'lang' => ['en', 'dv'],
        'type' => 'string',
        'displayType' => 'chip',
        'chip' => [
            'draft' => [
                'label' => ['en' => 'Draft', 'dv' => 'ڊرافٽ'],
                'color' => 'secondary',
                'prependIcon' => 'document-text',
            ],
            'submitted' => [
                'label' => ['en' => 'Submitted', 'dv' => 'سَبْمِٽِڊ'],
                'color' => 'primary',
                'prependIcon' => 'send',
            ],
            'rejected' => [
                'label' => ['en' => 'Rejected', 'dv' => 'ڕِجيڪْٽِڊ'],
                'color' => 'error',
                'prependIcon' => 'x-circle',
            ],
            'accepted' => [
                'label' => ['en' => 'Accepted', 'dv' => 'هَڪَرِدَسْڪِڊ'],
                'color' => 'success',
                'prependIcon' => 'check-circle',
            ],
        ],
    ],
```

Full example — integer values:

```
    'entry_type' => [
        'type' => 'number',
        'displayType' => 'chip',
        'chip' => [
            1 => ['label' => ['en' => 'Type 1', 'dv' => '1 ھَدِڤٮ'],
'color' => 'primary'],
            2 => ['label' => ['en' => 'Type 2', 'dv' => '2 ھَدِڤٮ'],
'color' => 'success'],
            3 => ['label' => ['en' => 'Type 3', 'dv' => '3 ھَدِڤٮ'],
'color' => 'warning'],
        ],
    ],
```

How CCS processes chips: When building headers, the CCS collapses the localized `label` within each chip option to a single string for the requested language. So the frontend receives:

```
    {
      "chip": {
        "draft": { "label": "Draft", "color": "secondary",
"prependIcon": "document-text" }
      }
    }
```

---

displayProps — Generic Display Properties

`displayProps` is an alternative to a named sub-key (like `chip`). It serves two purposes:

1. **As a fallback for `displayType`:** When `displayType` is `'chip'` but you want to use a flat structure instead of a separate `chip` key:'gender' => [
   'displayType' => 'chip',
   'displayProps' => [
      'M' => ['label' => 'Male', 'color' => 'primary', 'prependIcon' => 'user'],
      'F' => ['label' => 'Female', 'color' => 'success', 'prependIcon' => 'users'],
   ],
]
2. **For non-chip display types** that need extra config (e.g., checkbox styling):'is_active' => [
   'displayType' => 'checkbox',
   'displayProps' => [
      'label' => ' ',
      'color' => 'primary',
   ],
],

Resolution order: When building headers, the CCS checks for config in this order:

1. `$def[$displayType]` — a key matching the displayType name (e.g., `chip`)
2. `$def['displayProps']` — the generic fallback

If `displayType` is `'chip'` and you define a `chip` key, that takes priority over `displayProps`.

How displayType Resolution Works

When the CCS builds a header from a column definition, it:

1. Sets `header.displayType` to the value of `displayType`
2. Looks for a config object at `$def[$displayType]` (e.g., `$def['chip']`)
3. If not found, looks for `$def['displayProps']`
4. If found, normalizes the config (e.g., collapses localized labels for chips)
5. Attaches the config to the header under the displayType name (e.g., `header.chip = { ... }`)

Select & Dropdown Configuration

When a column needs a dropdown selector (for forms or inline editing), you configure it with `inputType: 'select'` and provide a `select` key (or `filterable` as a legacy alternative).

The select Key

The `select` key defines how dropdown options are sourced. It supports three **modes**: `self`, `relation`, and `url`.

Common structure:

```
    'column_name' => [
        'inputType' => 'select',
        'select' => [
            'type' => 'select',          // Optional identifier
            'label' => ['en' => '...'],  // Optional override label
            'mode' => 'self',            // Required: 'self',
'relation', or 'url'
            // ... mode-specific keys
            'itemTitle' => ...,          // Which field to display as
option text
            'itemValue' => '...',        // Which field to use as
option value
        ],
    ],
```

Mode: self

Options are defined inline in the schema. Use when you have a small, static list of options.

Required keys:

- `mode`: `'self'`
- `items`: Array of option objects
- `itemTitle`: Localized map or string — tells the CCS which field in each item holds the display text
- `itemValue`: String — tells the CCS which field in each item holds the value

```
    'sender_type' => [
        'type' => 'string',
        'inputType' => 'select',
        'select' => [
            'type' => 'select',
            'label' => ['en' => 'Sender Type', 'dv' => 'ؤ،ؤ،ؤ،'],
            'mode' => 'self',
            'items' => [
                ['itemTitleEn' => 'Person',       'itemTitleDv' =>
'ؤ،ؤ،ؤ،',       'itemValue' => 'person'],
                ['itemTitleEn' => 'Organization', 'itemTitleDv' =>
'ؤ،ؤ،ؤ،ؤ،ؤ،ؤ،', 'itemValue' => 'organization'],
                ['itemTitleEn' => 'Other',        'itemTitleDv' =>
'ؤ،ؤ،ؤ،ؤ،',     'itemValue' => 'other'],
            ],
            'itemTitle' => ['en' => 'itemTitleEn', 'dv' =>
'itemTitleDv'],
            'itemValue' => 'itemValue',
        ],
    ],
```

How it works: The CCS reads `itemTitle` for the current language (e.g., `'itemTitleEn'` for `lang=en`), then builds a pruned array with just the title and value fields from each item.

Items structure: Each item is an associative array. The keys should match what `itemTitle` and `itemValue` point to:

```
    [
```

```
        'itemTitleEn' => 'Display Text (English)',
        'itemTitleDv' => 'Display Text (Dhivehi)',
        'itemValue'   => 'actual_stored_value',
    ]
```

Empty/reset option: Include an item with an empty value to allow clearing the selection:

```
    ['itemTitleEn' => '-empty-', 'itemTitleDv' => '-ښَرُمْ-', 'itemValue'
=> ''],
```

Mode: relation

Options are fetched dynamically from a related model's GAPI endpoint. Use when options come from another database table via an Eloquent relationship.

Required keys:

- `mode`: `'relation'`
- `relationship`: The Eloquent relationship method name on the current model
- `itemTitle`: Localized map — column name on the **related** model that holds the display text
- `itemValue`: String — column name on the related model that holds the value (usually `'id'`)

Optional keys:

- `multiple`: `bool` — whether multi-select is allowed
- `sourceModel`: `string` — hint for the related model name (not required when `relationship` is provided)
- `value`: `string` — the column on the **current** model that stores the selected value

```
    'country_id' => [
        'hidden' => true,
        'type' => 'number',
        'inputType' => 'select',
        'select' => [
            'multiple' => true,
```

```
            'label' => ['dv' => 'ګڼډوګ', 'en' => 'Country'],
            'mode' => 'relation',
            'relationship' => 'country',
            'itemTitle' => [
                'dv' => 'name_div',
                'en' => 'name_eng',
            ],
            'itemValue' => 'id',
            'value' => 'country_id',
        ],
    ],
```

How it works: The CCS resolves the related model via the relationship, gets its class name, and generates a GAPI URL:

```
/api/gapi/Country?columns=id,name_eng&sort=name_eng&pagination=off&wrap
=data
```

The frontend then fetches this URL to populate the dropdown.

If `relationship` is missing: The CCS attempts to guess the related model name by stripping `_id` from the column key and converting to StudlyCase (e.g., `country_id` → `Country`). A validator warning is emitted.

Mode: url

Options are fetched from a custom API endpoint. Use when options come from an external API or a non-standard endpoint.

Required keys:

- `mode`: `'url'`
- `url`: The full URL to fetch options from

```
    'category' => [
        'inputType' => 'select',
        'select' => [
            'mode' => 'url',
```

```
            'url' => '/api/categories?format=select',
            'itemTitle' => 'name',
            'itemValue' => 'id',
        ],
    ],
```

How it works: The CCS passes the URL through directly to the output. The frontend is responsible for fetching and parsing the response.

If `url` is missing or empty: An `InvalidArgumentException` is thrown.

Common Select Sub-Keys

Key	Type	Required	Description
`mode`	`string`	Yes	`'self'`, `'relation'`, or `'url'`. Defaults to `'self'` if omitted.
`items`	`array`	For `self` mode	Array of option objects.
`itemTitle`	`string` or `array`	Recommended	Field name (or lang map of field names) for display text.
`itemValue`	`string`	Recommended	Field name for the option value. Defaults to the column `key`.
`label`	`string` or `array`	No	Override label for the dropdown.
`type`	`string`	No	Meta identifier, typically `'select'`.
`relationship`	`string`	For `relation` mode	Eloquent relationship method name.
`url`	`string`	For `url` mode	The URL to fetch options from.
`multiple`	`bool`	No	Allow multiple selections.
`value`	`string`	No	Column on the current model that stores the value.
`sourceModel`	`string`	No	Hint for the related model class name.

Language Support

How lang Filtering Works

When a request includes `?lang=en`, the CCS filters out columns that do not support that language. The process:

1. For each column token, look up the column definition
2. Read the `lang` array (e.g., `['en', 'dv']`)
3. Normalize to lowercase and deduplicate
4. Check if the requested language is in the array
5. If yes → include the column; if no → exclude it

This filtering applies to:

- **Headers** — only matching columns get a header
- **Data links** — only matching columns appear in the `columns` query parameter
- **Form fields** — only matching columns appear as fields
- **Filters** — only matching columns appear as filter inputs

Columns Without lang

If a column definition has no `lang` key, or `lang` is not an array, the column is included in **all** languages. This is the safe default but may lead to mismatched language content appearing.

Language-Specific Columns

A common pattern for multilingual data stores is to have separate columns per language:

```
'first_name_eng' => [
    'label' => ['en' => 'First Name'],
    'lang' => ['en'],           // Only in English views
],
'first_name_div' => [
    'label' => ['dv' => 'ﬁﷺﷺﷺﷺ ﷺﷺﷺﷺ'],
    'lang' => ['dv'],           // Only in Dhivehi views
],
```

When `lang=en` is requested, only `first_name_eng` appears. When `lang=dv` is requested, only `first_name_div` appears.

lang and Header Language Override

The CCS also uses the `lang` array to determine a "language override" value attached to each header. This tells the frontend which **other** language the column could support:

- If `lang` is `['en', 'dv']` and request is `lang=en` → `header.lang = 'dv'`
- If `lang` is `['en']` only → no `header.lang` override (since there is no other language)

This allows the frontend to offer a "switch language" toggle on a per-column basis.

Relation Columns

Relation columns use dot-notation (e.g., `country.name_eng`) and appear in the `columns` array of view configs, not directly in `apiSchema()`. However, the related model's `apiSchema()` is resolved by the CCS to build headers and filters for these columns.

For example, if a view config includes `"columns": ["id", "country.name_eng"]`, the CCS will:

1. Detect `country.name_eng` as a relation column
2. Resolve the `country` relationship on the parent model
3. Find the `Country` model's `apiSchema()` → `columns.name_eng`
4. Use that definition for the header (title, sortable, displayType, etc.)

relationLabel

Property	Value
Type	`string` or `array` (localized map)
Required	No

Property	Value
Default	Falls back to `label`

When a column from this model is used as a **relation column** in another model's view (via dot-notation), `relationLabel` provides an alternative label for that context.

```
    // In Country model's apiSchema:
    'name_eng' => [
        'label' => ['en' => 'Name'],                        // Used when
showing Country directly
        'relationLabel' => ['en' => 'Country', 'dv' => 'ءَمُڻُ'],  // Used
when shown as relation column
    ],
```

When another model's view has `country.name_eng`, the header title will be `"Country"` (from `relationLabel`) instead of `"Name"` (from `label`).

If missing: The CCS falls back to the regular `label`.

Computed Attributes

Computed attributes (Eloquent `Attribute` accessors) can be defined in `apiSchema()` just like regular database columns. The attribute name in the `columns` map should be the **snake_case** version of the accessor method name.

```
    class Person extends BaseModel
    {
        protected $appends = ['full_name'];

        protected array $computedAttributeDependencies = [
            'full_name' => ['first_name_eng', 'middle_name_eng',
'last_name_eng'],
        ];

        protected function fullName(): Attribute
        {
            return Attribute::make(
                get: fn () => collect([$this->first_name_eng, $this-
>middle_name_eng, $this->last_name_eng])
                    ->filter()->join(' ')
```

```
            );
        }

        public function apiSchema(): array
        {
            return [
                'columns' => [
                    'full_name' => [
                        'hidden' => false,
                        'key' => 'full_name',
                        'label' => ['en' => 'Full Name', 'dv' =>
'ﷺ'],
                        'type' => 'string',
                        'displayType' => 'englishText',
                        'lang' => ['en', 'dv'],
                    ],
                    // ... other columns
                ],
            ];
        }
    }
```

Key points:

- Add the attribute to `$appends` so it is included in JSON responses
- Define `$computedAttributeDependencies` so the query auto-selects the required database columns even when they're not explicitly requested
- Dependency columns auto-selected this way are hidden from JSON unless explicitly requested in the `columns` query parameter

---

Complete Annotated Example

A full model demonstrating all key types:

```
    <?php

    namespace App\Models;

    use Ogp\UiApi\Models\BaseModel;
    use Illuminate\Database\Eloquent\Casts\Attribute;
    use Illuminate\Validation\Rule;

    class Invoice extends BaseModel
    {
        protected $table = 'invoices';
```

```
        protected array $searchable = ['id', 'invoice_number',
'status'];

        protected $appends = ['display_name'];

        protected array $computedAttributeDependencies = [
            'display_name' => ['invoice_number', 'status'],
        ];

        protected function casts(): array
        {
            return [
                'id' => 'integer',
                'invoice_number' => 'string',
                'amount' => 'decimal:2',
                'is_paid' => 'boolean',
                'status' => 'string',
                'customer_id' => 'integer',
                'issued_at' => 'datetime',
                'created_at' => 'datetime',
            ];
        }

        protected function displayName(): Attribute
        {
            return Attribute::make(
                get: fn () => "#{$this->invoice_number} ({$this-
>status})"
            );
        }

        public function apiSchema(): array
        {
            return [
                'columns' => [

                    // ─── Hidden primary key ───
                    'id' => [
                        'hidden' => true,
                        'key' => 'id',
                        'label' => ['en' => 'ID'],
                        'lang' => ['en', 'dv'],
                        'type' => 'integer',
                        'displayType' => 'text',
                        'sortable' => true,
                    ],

                    // ─── Computed attribute ───
                    'display_name' => [
                        'hidden' => false,
                        'key' => 'display_name',
                        'label' => ['en' => 'Invoice'],
                        'lang' => ['en'],
                        'type' => 'string',
                        'displayType' => 'text',
                    ],

                    // ─── Basic text field ───
```

```
                    'invoice_number' => [
                        'hidden' => false,
                        'key' => 'invoice_number',
                        'label' => ['en' => 'Invoice #', 'dv' =>
'ބަޔަކަށް'],
                        'lang' => ['en', 'dv'],
                        'type' => 'string',
                        'displayType' => 'text',
                        'inputType' => 'textField',
                        'formField' => true,
                        'sortable' => true,
                    ],

                    // ─── Numeric field ───
                    'amount' => [
                        'hidden' => false,
                        'key' => 'amount',
                        'label' => ['en' => 'Amount', 'dv' => 'ފަރާތް'],
                        'lang' => ['en', 'dv'],
                        'type' => 'number',
                        'displayType' => 'text',
                        'inputType' => 'numberField',
                        'formField' => true,
                        'sortable' => true,
                    ],

                    // ─── Boolean with checkbox display ───
                    'is_paid' => [
                        'hidden' => false,
                        'key' => 'is_paid',
                        'label' => ['en' => 'Paid', 'dv' => 'ފައިސާ ދައްކާ'],
                        'lang' => ['en', 'dv'],
                        'type' => 'boolean',
                        'displayType' => 'checkbox',
                        'inputType' => 'checkbox',
                        'formField' => true,
                        'sortable' => true,
                        'displayProps' => [
                            'label' => ' ',
                            'color' => 'success',
                        ],
                    ],

                    // ─── Chip display + self-mode select ───
                    'status' => [
                        'hidden' => false,
                        'key' => 'status',
                        'label' => ['en' => 'Status', 'dv' => 'ހާލަތު'],
                        'lang' => ['en', 'dv'],
                        'type' => 'string',
                        'displayType' => 'chip',
                        'inputType' => 'select',
                        'formField' => true,
                        'inlineEditable' => true,
                        'sortable' => true,
                        'chip' => [
                            'pending' => [
                                'label' => ['en' => 'Pending', 'dv' =>
```

```
'ﷺ'],
                                'color' => 'warning',
                                'prependIcon' => 'clock',
                            ],
                            'paid' => [
                                'label' => ['en' => 'Paid', 'dv' =>
'ﷺ'],
                                'color' => 'success',
                                'prependIcon' => 'check-circle',
                            ],
                            'overdue' => [
                                'label' => ['en' => 'Overdue', 'dv' =>
'ﷺ'],
                                'color' => 'error',
                                'prependIcon' => 'exclamation-circle',
                            ],
                        ],
                        'select' => [
                            'type' => 'select',
                            'mode' => 'self',
                            'items' => [
                                ['itemTitleEn' => 'Pending',
'itemTitleDv' => 'ﷺ', 'itemValue' => 'pending'],
                                ['itemTitleEn' => 'Paid',
'itemTitleDv' => 'ﷺ',  'itemValue' => 'paid'],
                                ['itemTitleEn' => 'Overdue',
'itemTitleDv' => 'ﷺ', 'itemValue' => 'overdue'],
                            ],
                            'itemTitle' => ['en' => 'itemTitleEn', 'dv'
=> 'itemTitleDv'],
                            'itemValue' => 'itemValue',
                        ],
                    ],

                    // ─── Foreign key with relation-mode select ───
                    'customer_id' => [
                        'hidden' => true,
                        'key' => 'customer_id',
                        'label' => ['en' => 'Customer', 'dv' =>
'ﷺ'],
                        'lang' => ['en', 'dv'],
                        'type' => 'number',
                        'displayType' => 'text',
                        'inputType' => 'select',
                        'formField' => true,
                        'select' => [
                            'mode' => 'relation',
                            'relationship' => 'customer',
                            'itemTitle' => ['en' => 'name_eng', 'dv' =>
'name_div'],
                            'itemValue' => 'id',
                            'value' => 'customer_id',
                        ],
                    ],

                    // ─── Date field ───
                    'issued_at' => [
                        'hidden' => false,
```

```
                        'key' => 'issued_at',
                        'label' => ['en' => 'Issued Date', 'dv' =>
'ؤَمَّرِزْ'],
                        'lang' => ['en', 'dv'],
                        'type' => 'datetime',
                        'displayType' => 'date',
                        'inputType' => 'datepicker',
                        'formField' => true,
                        'sortable' => true,
                    ],

                    // ─── Timestamp (not in forms) ───
                    'created_at' => [
                        'hidden' => true,
                        'key' => 'created_at',
                        'label' => ['en' => 'Created At'],
                        'lang' => ['en', 'dv'],
                        'type' => 'datetime',
                        'displayType' => 'date',
                        'formField' => false,
                    ],
                ],
                'searchable' => $this->searchable,
            ];
        }

        // ─── Relationships ───

        public function customer()
        {
            return $this->belongsTo(Customer::class);
        }

        // ─── Validation ───

        public static function baseRules(): array
        {
            return [
                'invoice_number' => ['required', 'string', 'max:50'],
                'amount' => ['required', 'numeric', 'min:0'],
                'is_paid' => ['nullable', 'boolean'],
                'status' => ['required', 'string',
'in:pending,paid,overdue'],
                'customer_id' => ['required', 'integer',
'exists:customers,id'],
                'issued_at' => ['required', 'date'],
            ];
        }

        public static function rulesForCreate(): array
        {
            $rules = static::baseRules();
            $rules['id'] = ['sometimes', 'integer',
Rule::unique('invoices', 'id')];
            return $rules;
        }

        public static function rulesForUpdate(?int $id = null): array
        {
```

```
            $rules = static::baseRules();
            $rules['id'] = $id !== null
                ? ['required', 'integer', Rule::unique('invoices',
'id')->ignore($id)]
                : ['required', 'integer'];
            return $rules;
        }
    }
```

Quick Reference Table

Column Definition Keys

Key	Type	Required	Default	Used By	Purpose
`hidden`	`bool`	No	`false`	Headers	Hide column from table headers
`key`	`string`	No	Array key	Headers, Forms, Filters	Identifier in output payload
`label`	`string|array`	No	Auto from name	Headers, Forms, Filters	Human-readable display name
`type`	`string`	No	`'string'`	Forms, Filters, Headers	Data type; drives default inputType
`lang`	`array`	No	All langs	All	Language availability filter
`sortable`	`bool`	No	`false`	Headers	Enable column sorting
`displayType`	`string`	No	*(omitted)*	Headers	Frontend rendering mode
`inputType`	`string`	No	From `type`	Forms, Filters	Form input component type
`formField`	`bool`	No	`false`	Forms	Include in auto-generated forms
`inlineEditable`	`bool`	No	`false`	Headers	Allow inline table editing

Key	Type	Required	Default	Used By	Purpose
`fieldComponent`	`string`	No	*(omitted)*	Pass-through	Frontend form component hint
`validationRule`	`string`	No	*(omitted)*	Pass-through	Client-side validation hint
`chip`	`array`	When displayType =chip	*(none)*	Headers	Chip value-to-style mapping
`displayProps`	`array`	No	*(none)*	Headers	Generic display config (fallback for chip, checkbox, etc.)
`select`	`array`	When inputType=select	*(none)*	Forms, Filters	Dropdown options configuration
`filterable`	`array`	No	*(none)*	Filters (legacy)	Filter-specific configuration
`relationLabel`	`string |array`	No	*(none)*	Headers (relation)	Label override when used as a relation column

displayType Values

Value	Additional config needed	Notes
`'text'`	No	Default plain text
`'englishText'`	No	LTR text rendering
`'date'`	No	Date formatting
`'checkbox'`	Optional `displayProps`	Boolean display
`'chip'`	`chip` or `displayProps`	Value-mapped colored badges
`'select'`	`select` or `displayProps`	Select-style display

inputType Values

Value	Typical `type`	Notes
`'textField'`	`string`	Default for strings
`'numberField'`	`number`, `integer`	Default for numbers
`'datepicker'`	`date`, `datetime`	Default for dates
`'checkbox'`	`boolean`	Default for booleans

Value	Typical `type`	Notes
`'select'`	Any	Requires `select` or `filterable` config

Select Modes

Mode	Options source	Required keys
`'self'`	Inline static list	`items`, `itemTitle`, `itemValue`
`'relation'`	Related model GAPI	`relationship`, `itemTitle`, `itemValue`
`'url'`	Custom endpoint	`url`, `itemTitle`, `itemValue`

Validation Checklist

Before shipping a new model's `apiSchema()`, verify:

- Every column has `type` and `displayType` set
- Every column has a `lang` array
- Columns with `displayType: 'chip'` have a matching `chip` or `displayProps` key
- Columns with `inputType: 'select'` have a `select` or `filterable` block
- Select blocks with `mode: 'self'` have non-empty `items`
- Select blocks with `mode: 'relation'` have a `relationship` value
- Select blocks with `mode: 'url'` have a valid `url` string
- Columns that should appear in forms have `formField: true`
- `label` uses localized map format for multi-language support
- Chip maps cover **all** possible values for the column
- `key` matches the array key for consistency
- The model extends `Ogp\UiApi\Models\BaseModel`

> **Tip:** Run the `ViewConfigValidator` against your view config — it will catch many of these issues automatically.